DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING
Course Code: CS-116
Course Title: Object Oriented Programming
Complex Engineering Problem
FE Batch 2025, Spring Semester 2026
Grading Rubric
TERM PROJECT

Student No.	Name	Roll No.
S1	Muhammad Ahsan	CS-125
S2	Muhammad Ubaid Ullah	CS-128
S3	Muhammad Mutahhar Khan	CS-131
S4	Muhammad Hasan	CS-132

CRITERIA AND SCALES				Marks Obtained			
				S1	S2	S3	S4
Criterion 1: Does the application meet the desired specifications and produce the desired outputs? (CPA-1, CPA-3)							
1	2	3	4				
The application does not meet the desired specifications and is producing incorrect outputs.	The application partially meets the desired specifications and is producing incorrect or partially correct outputs.	The application meets the desired specifications but is producing incorrect or partially correct outputs.	The application meets all the desired specifications and is producing correct outputs.				
Criterion 2: How well is the code organization?							
1	2	3	4				
The code is poorly organized and very difficult to read.	The code is readable only to someone who knows what it is supposed to be doing.	Some part of the code is well organized, while some part is difficult to follow.	The code is well organized and very easy to follow.				
Criterion 3: How friendly is the application interface? (CPA-1, CPA-3)							
1	2	3	4				
The application interface is difficult to understand and use.	The application interface is easy to understand and but not that comfortable to use.	The application interface is very easy to understand and use.	The application interface is very interesting/ innovative and easy to understand and use.				
Criterion 4: How does the student performed individually and as a team member? (CPA-2, CPA-3)							
1	2	3	4				
The student did not work on the assigned task.	The student worked on the assigned task, and accomplished goals partially.	The student worked on the assigned task, and accomplished goals satisfactorily.	The student worked on the assigned task, and accomplished goals beyond expectations.				
Criterion 5: Does the report adhere to the given format and requirements?							
1	2	3	4				
The report does not contain the required information and is formatted poorly.	The report contains the required information only partially but is formatted well.	The report contains all the required information but is formatted poorly.	The report contains all the required information and completely adheres to the given format.				
			Total Marks:				

Table Of Contents

INDIVIDUAL CONTRIBUTIONS

Muhammad Hasan

Muhammad Hasan led the user authentication module and core borrow/return system:

- Designed and implemented the User, Student, and Librarian class hierarchy with proper access control.
- Built the complete login and student registration system including seat number format validation (XX-XXXXX).
- Implemented borrowResource() and returnResource() with daily borrow limit enforcement (max 2 per day).
- Implemented fine calculation: Rs. 10 per late day plus Rs. 50 optional damage fee.
- Handled file I/O for users.txt and borrows.txt using fstream and stringstream.

Muhammad Ahsan

Muhammad Ahsan built the resource management system and all librarian functions:

- Designed the abstract Resource class and derived Book and Magazine classes.
- Implemented addResource(), deleteResource(), updateResource(), viewAllResources() and AccountInfo().
- Built issuedReport() and printStudentReport() for the librarian panel.
- Handled polymorphic resource loading from file using type-based factory object creation.
- Managed file I/O for resources.txt with proper serialization.

Muhammad Ubaidullah

Muhammad Ubaidullah handled the reservation system, account management, and records:

- Designed LibraryRecord base class and derived classes BorrowRecord and Reservation.
- Implemented reserveResource(), showMyReservations(), and automatic reservation cancellation on borrow.
- Built the Account class with payFine(), recharge(), fineProcessing(), and display().
- Managed file I/O for reservations.txt.
- Wrote and executed test cases to verify integration between borrow and reservation modules.

Muhammad Mutahhar

Muhammad Mutahhar built the date system, main menu, and handled final integration:

- Designed and implemented the Date class with leap year handling and Tdays() for date difference.
- Built App.cpp with complete student and librarian menus along with proper exception handling.
- Added console color output using Windows API (SetConsoleTextAttribute).
- Performed full integration testing across all modules and resolved bugs.
- Compiled the project report and class diagrams.

CHAPTER 1: INTRODUCTION

1.1 Problem Description

Managing a library manually is slow and error-prone. Tracking book availability, who borrowed what, handling fine calculations, and managing reservations by hand leads to mistakes and wasted time for both students and librarians.

This project implements a Library Management System in C++ using Object Oriented Programming concepts. All data is stored in text files and the system provides a menu-driven interface for two types of users:

- Librarian — manages resources, views reports, and monitors student activity.
- Student — browses resources, borrows and returns books/magazines, makes reservations, pays fines, and manages account balance.

1.2 Project Motivation, Relevance, and Beneficiaries

The motivation behind this project is to apply OOP concepts learned in CS-116 to a real-world problem. Manual library systems in educational institutions suffer from misplaced records, slow processing, and difficulty tracking fines and reservations.

Beneficiaries:

- Students — quickly check availability, borrow, reserve or return items, and track their own history and fines.
- Librarians — structured system to manage inventory, generate reports, and monitor borrowing activity.
- Institution — reduced manual errors, better resource control, and improved student experience.

CHAPTER 2: OOP FEATURES IMPLEMENTED

2.1 Encapsulation

Encapsulation is used in the Account class where all financial data is kept private and accessed through getter and setter methods. The Resource class also keeps title, author, and copies protected with public methods to access them. The User class keeps password protected and never exposes it directly only verifies it through **checkPassword().**

2.2 Inheritance

Book and **Magazine** inherit from **Resource**, and **Student** and **Librarian** inherit from **User**. **BorrowRecord** and **Reservation** both inherit from **LibraryRecord**. This way common attributes and methods are written once in the parent and reused in all child classes.

2.3 Polymorphism (Runtime)

Resource declares virtual functions: **input()**, **display()**, **getIdentifier()**, **getType()**, and **setIdentifier()**. Book overrides these to work with ISBN, while Magazine overrides them to work with ISSN. **Library_System** stores Resource* pointers and the correct version is called at runtime depending on the actual object type.

2.4 Abstract Classes and Pure Virtual Functions

Resource has **getIdentifier()=0**, **getType()=0**, and **setIdentifier()=0** as pure virtual functions, making it abstract it cannot be instantiated directly. User has **displayRole()=0** as a pure virtual function. Both enforce a contract that all derived classes must implement these methods.

2.5 Composition

User contains an **Account** object, and **BorrowRecord** contains three **Date** objects these inner objects are created as part of the outer object and destroyed with it. App contains a **LibraryManagementSystem** object, and **LibraryManagementSystem** contains a **Library_System** object as direct members. **Library_System** also holds **vector<BorrowRecord>** and **vector<Reservation>** as actual objects stored directly inside it not as pointers. All of these inner objects cannot exist independently and are automatically destroyed when their outer class is destroyed.

2.6 Aggregation

Aggregation means one class uses objects of another class, but those objects can exist independently without the outer class. Unlike composition, destroying the outer object does not destroy the inner ones.

In this project, **Library_System** holds a **vector<Resource*>** — pointers to Book and Magazine objects that are created separately using `new` and just stored/referenced by **Library_System**. These Resource objects exist on their own and are not tied to the life of **Library_System** itself. Similarly, **LibraryManagementSystem** holds a **vector<User*>** where User objects are created independently and

simply managed by the class. This is aggregation because the users have their own existence outside the manager class.

2.7 Exception Handling

The system uses try-catch blocks throughout app.cpp and library functions. std::runtime_error is thrown for conditions like invalid login, borrow limit exceeded, and insufficient fine balance. std::invalid_argument is thrown for invalid inputs like negative recharge amounts.

2.8 File Handling

All data is saved to and loaded from four text files: users.txt, resources.txt, borrows.txt, and reservations.txt. The system uses fstream for reading and writing, and stringstream with pipe-delimited (|) parsing for object reconstruction on program startup.

2.9 STL Vectors

std::vector is used throughout to dynamically manage users, resources, borrow records, and reservations. This avoids fixed-size arrays and allows the system to handle any number of entries at runtime.

CHAPTER 3: CLASS DIAGRAM

3.1 Flow of the Project

Step 1: Program Start When the program starts, the main menu is displayed with the following options:

1. Register Student
2. Login
3. Exit

Step 2: Register Student If the user selects Register Student, the program asks the student to enter the following details:

- First Name
- Last Name
- Department
- Username
- Password
- Balance

After entering all required information, the student account is successfully created and stored in the system.

Step 3: Login If the user selects Login, the system asks the user to choose the login type:

1. Login as Student
2. Login as Librarian

The user must enter Username and Password. If credentials are correct, login is successful. If a student account is used to log in as a librarian, the system displays an error message.

Step 4: Student Menu After successful student login, the system displays the following options:

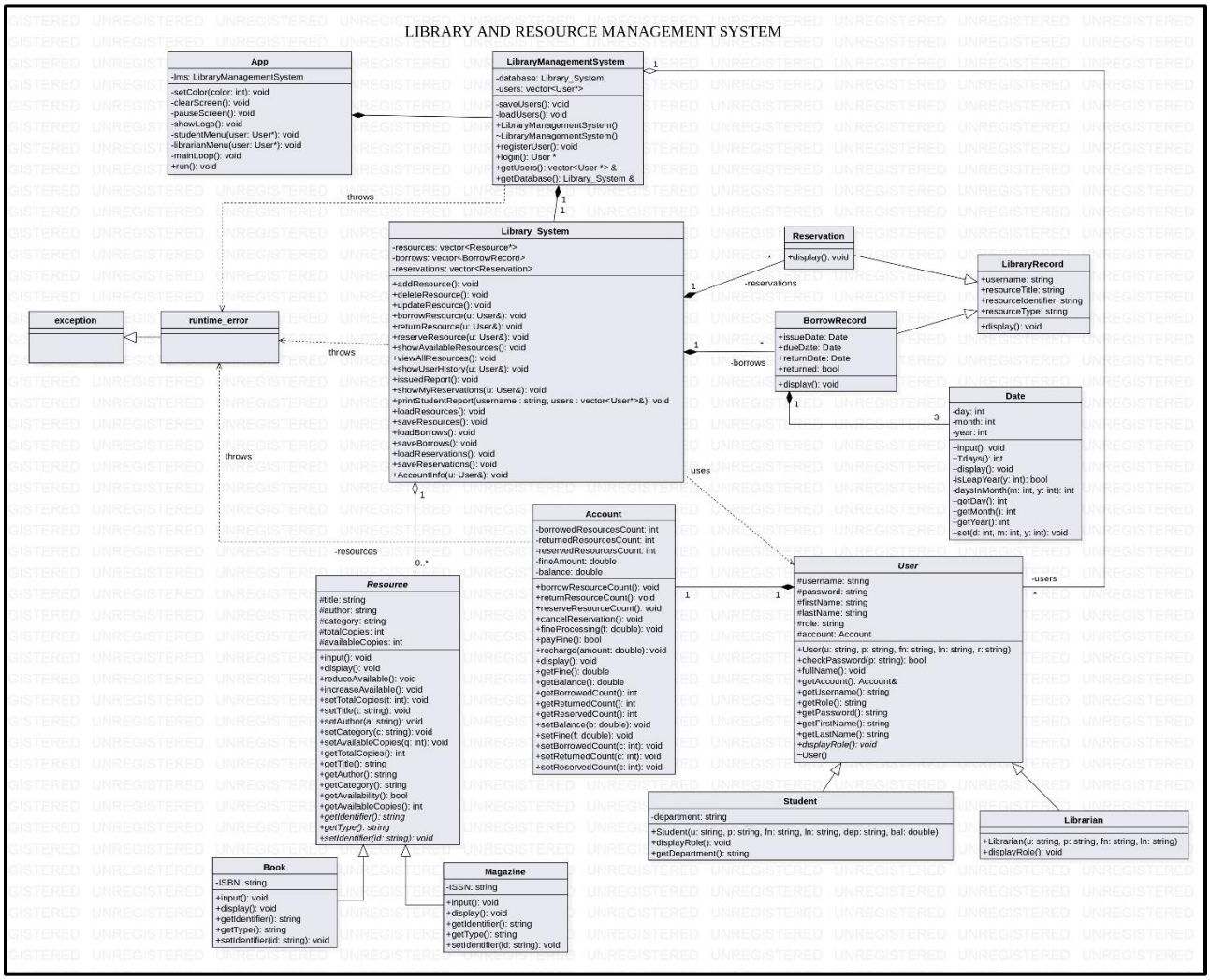
1. View Resources
2. Borrow Resources
3. Return Resources
4. Reserve Resources
5. My History
6. My Reservations
7. Pay Fine
8. Recharge Balance
9. Account Info
10. Logout

Step 5: Librarian Menu After successful librarian login, the system displays the following options:

1. Add Resource
2. Update Resource
3. Delete Resource
4. View All Resources
5. Issued Report
6. Student Report
7. Logout

Step 6: Exit When the user selects Exit from the main menu, all data is automatically saved to text files and the program closes.

3.2 Class Diagram



CHAPTER 4: MOST CHALLENGING PARTS

4.1 Polymorphic Resource Loading from File

Library_System stores a vector<Resource*>. When loading from resources.txt, we need to read the type field first, then create either new Book() or new Magazine() before casting to Resource*. Getting this factory-style loading right especially combined with pipe-delimited parsing was the most technically difficult part of the project.

4.2 Date Serialization in BorrowRecord

BorrowRecord contains three Date objects: issueDate, dueDate, and returnDate. Each Date has day, month, and year. Saving all three as integers with pipes and loading them back in exactly the right order using stringstream required careful counting of ignore() calls. One wrong call shifted all subsequent data and corrupted records.

4.3 Fine Calculation with Tdays()

The Date class Tdays() function converts any date into a total day count from a base point. Subtracting two Tdays() values gives the number of days between dates. Correctly handling leap years inside this function — including the 400-year and 100-year rules required careful implementation and testing.

4.4 Designing the Inheritance Hierarchy

Deciding what logic belongs in the abstract base class versus the derived class required planning. For Resource, we put common attributes (title, author, copies) and shared behavior in the base, while ISBN/ISSN and type-specific display go in Book and Magazine respectively.

4.5 Reservation Priority and Conflict Prevention

The reservation logic required handling multiple conflicts simultaneously — blocking borrowing if another user had a prior reservation, preventing duplicates, and auto-removing reservations once the reserving user borrowed the resource. Coordinating these checks across the borrows vector, reservations vector, and Account object at the same time was the most challenging part.

CHAPTER 5: NEW THINGS LEARNED IN C++

1. **Abstract classes and pure virtual functions** — We understood how to design a class that cannot be instantiated and forces derived classes to implement required behavior.

2. **Runtime polymorphism via vtable** — We saw how a base class pointer calls the correct overridden function at runtime without any if/else based on type.

3. **Exception handling with custom messages** — We learned the difference between runtime_error and invalid_argument and how to propagate exceptions up to the caller.

4. **Memory management with vector<Pointer*>** — We understood how to manage heap-allocated objects in a vector and why the destructor must explicitly delete each pointer.

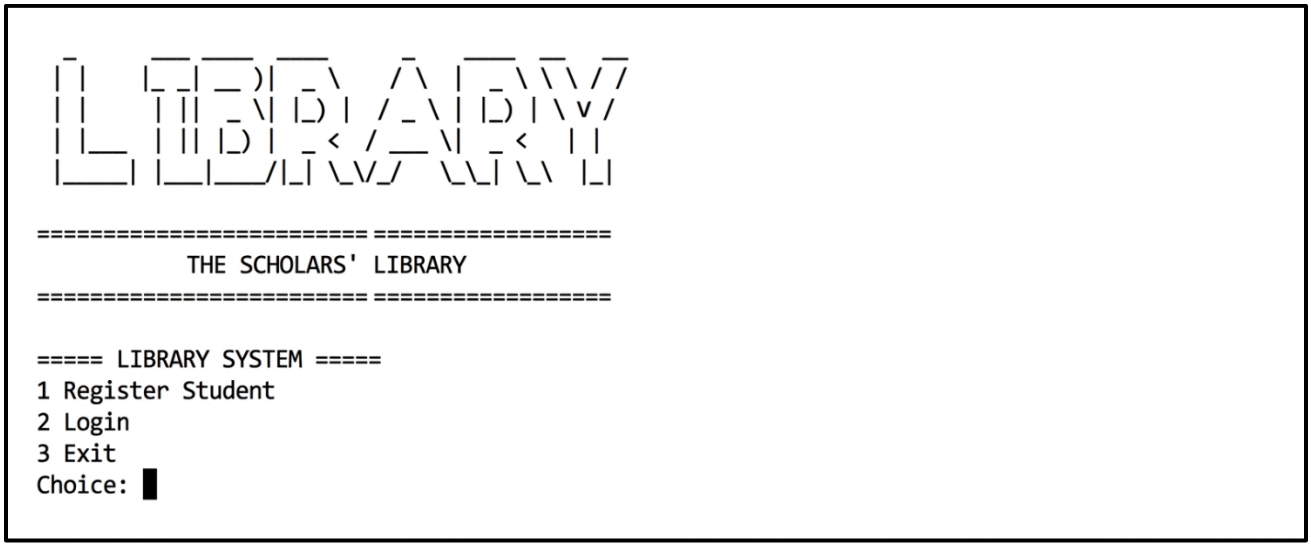
5. **File serialization with stringstream** — We learned to convert class objects into pipe-delimited strings and reconstruct them using getline with a custom delimiter.

6. **Composition vs Inheritance decision** — We understood when to embed an object as a member (User contains Account) versus when to inherit (Student extends User).

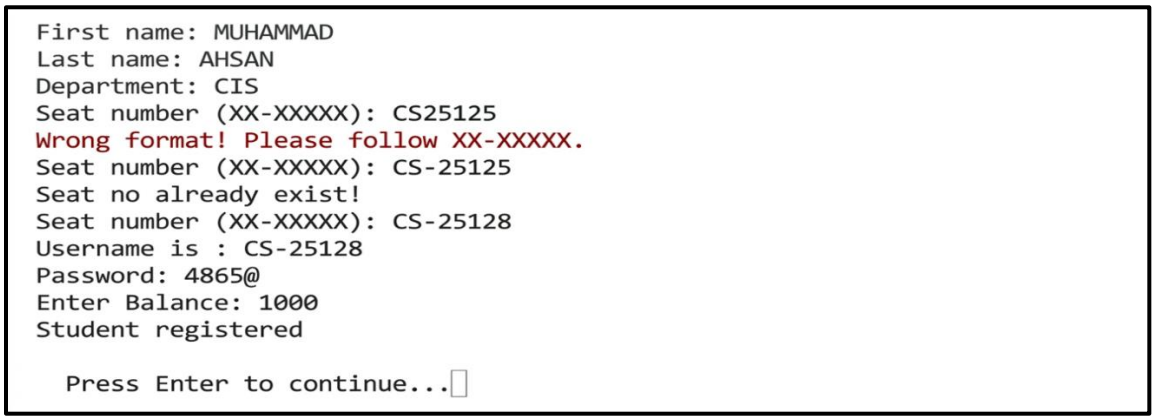
CHAPTER 6: TEST CASES

Test Case 1: Student Registration & Login	
Objective	Verify that a student can register with valid details, log in successfully, and access menus based on the selected role.
Steps	1. Register the student with required details: First Name: MUHAMMAD, Last Name: AHSAN, Dept: CIS, Seat: CS-25125, Password: 4865@, Balance: 1000. 2. Select the role (**Student** or **Librarian**). 3. Login using the same credentials.
Expected	Registration success message is displayed. Login succeeds successfully. Role selection options (**Student** / **Librarian**) appear, and the system displays the menu according to the selected role.

Interface



Registering the student:



Role Selection

```
===== LOGIN AS =====
1 Student
2 Librarian
Choice:
```

Student Login Credentials

```
Username: CS-25128
Password: 4865@
```

Student Menu Displays:

```
+======================================+
|            STUDENT MENU              |
+======================================+
|  Welcome : MUHAMMAD AHSAN
+--------------------------------------+
|  [1]  View Resources                 |
|  [2]  Borrow Resource                |
|  [3]  Return Resource                |
|  [4]  Reserve Resource               |
|  [5]  My History                     |
|  [6]  My Reservations                |
|  [7]  Pay Fine                       |
|  [8]  Recharge Balance               |
|  [9]  Account Info                   |
|  [10] Logout                         |
+--------------------------------------+
  >>
```

Test Case 2: Borrow Book and Return with Late Fine	
Objective	Verify borrowing reduces available copies and a late return applies the correct fine.
Steps	1. Login as student. 2. Borrow a book, 3. Return the book 1 day after the due date (8th day) with damage fee applied. 4. Check account fine.
Expected	Available copies decrease on borrow. On return, system calculates fine = 1 day x $10 = $10 and the damaged fee as well and adds it to account.

```
Today's Date: 7-5-2026

What would you like to borrow?
1 -> Book
2 -> Magazine
Choice: 1

How would you like to search?
1 -> Search by ISBN
2 -> Search by Author
3 -> Search by Title
Choice: 1
Enter ISBN: 978-149190399

--- Search Results ---
1.
----- Book -----
ISBN: 978-149190399

==============================
         RESOURCE DETAILS
==============================
Title        : The Pragmatic Programmer
Author       : Andrew Hunt & David Thomas
Category     : Programming
Availability : Available
Available Copies: 4 out of 5
==============================
Enter the number to borrow (0 to cancel): 1
Due Date: 14-5-2026

book Borrowed Successfully
```

```
Enter ISBN / ISSN: 978-149190399
Enter Return Date:
Day: 15
Month: 5
Year: 2026
Is The Resource Damaged?
1.Yes
2.No
1
Late By 1 Day(s)
Fine = 60       Added To Your Account
Returned Successfully
```

```
=========================
     Account Details
=========================

Name :MUHAMMAD AHSAN
Username: CS-25128
Resources currently borrowed: 0
Resources returned to date  : 1
Books reserved              : 0
Pending Fines               : $60
Current Account Balance     : $1000

Press Enter to continue...
```

Test Case 3: Borrow Limit Enforcement (Max 2/day)	
Objective	Verify the system prevents borrowing more than 2 resources on the same day.
Steps	1. Login as student. 2. Borrow first resource — succeeds. 3. Borrow second resource — succeeds. 4. Attempt to borrow a third resource on the same day.
Expected	Third borrow attempt throws: [Error]: Borrow limit reached! Max 2 resources per day.

```
----- Magazine -----
ISSN: 1356-7489

==============================
         RESOURCE DETAILS
==============================
Title        : PC Gamer
Author       : Future PLC
Category     : Gaming
Availability : Available
Available Copies: 10 out of 12
==============================
Enter the number to borrow (0 to cancel): 1
Resource borrowed successfully!
Enter Due Date
Day: 26
Month: 4
Year: 2026
magazine Borrowed Successfully
```

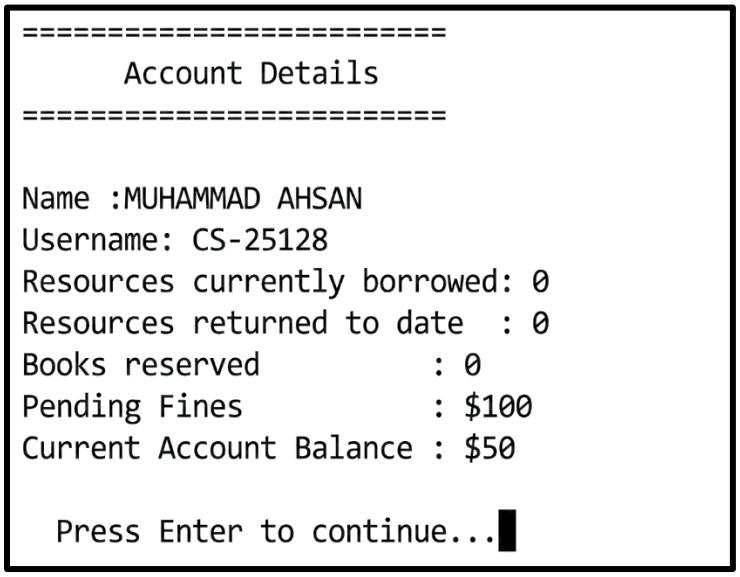
```
----- Book -----
ISBN: 978-013235088

==============================
         RESOURCE DETAILS
==============================
Title        : Clean Code
Author       : Robert C. Martin
Category     : Development
Availability : Available
Available Copies: 3 out of 4
==============================
Enter the number to borrow (0 to cancel): 1
Resource borrowed successfully!
Enter Due Date
Day: 25
Month: 4
Year: 2026
book Borrowed Successfully
```

Third book borrow shows an ERROR:

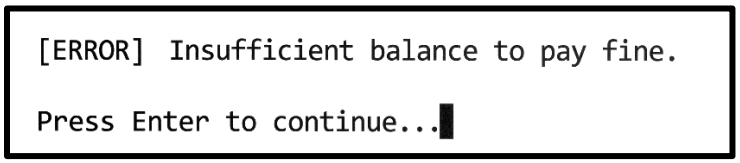
```
Today's Date: 24-4-2026

[ERROR] Borrow limit reached! Max 2 resources per day.

Press Enter to continue...
```

Test Case 4: Pay Fine with Insufficient Balance	
Objective	Verify the system correctly handles fine payment when account balance is insufficient.
Steps	1. Student has $50 balance and $100 pending fine. 2. Select Pay Fine from student menu.
Expected	[Error]: Insufficient balance to pay fine. Fine remains unchanged at $100.

Insufficient balance to pay fine:



Throws An Error



Test Case 5: Librarian Add, View, Delete Resource And View Student Report	
Objective	Verify the librarian can add a book, view it, delete it and generate a student report.
Steps	1. Login as librarian (admin/admin). 2. Add Resource -> Book -> enter ISBN, title, author, category, copies. 3. View All Resources — confirm book appears. 4. Delete Resource by ISBN/ISSN. 5. View Student Report (By His/Her Username)
Expected	Book appears after adding. After deletion, the book no longer appears in the resource list.

Librarian Menu

```
+--------------------------------------+
|           LIBRARIAN MENU             |
+--------------------------------------+
|  [1]  Add Resource                   |
|  [2]  Update Resource                |
|  [3]  Delete Resource                |
|  [4]  View All Resources             |
|  [5]  Issued Report                  |
|  [6]  Student Report                 |
|  [7]  Logout                         |
+--------------------------------------+
  >>
```

Adding Book & Viewing Resource:

```
1 -> Book
2 -> Magazine
Enter your choice: 1
ISBN: 978-1260226409
Title: Fundamentals of Electric Circuits
Author: Charles K. Alexander and Matthew N. O. Sadiku
Category: Education
Total Copies: 10

Resource added successfully!

  Press Enter to continue...
```

```
---- All Resources ----

----- Book -----
ISBN: 978-013110362

==============================
         RESOURCE DETAILS
==============================
Title        : The C Programming Language
Author       : Kernighan & Ritchie
Category     : Education
Availability : Available
Available Copies: 3 out of 5
==============================

----- Book -----
ISBN: 978-013225088

==============================
         RESOURCE DETAILS
==============================
Title        : Clean Code
Author       : Robert C. Martin
Category     : Development
Availability : Available
Available Copies: 2 out of 4
==============================
```

Deleting Resource By Title

```
Enter title to delete: Fundamentals of Electric Circuits
Resource deleted

  Press Enter to continue...
```

Viewing Student Report

```
Enter student username: CS-25125

====================================
       OFFICIAL STUDENT REPORT
====================================
Student ID: CS-25125
Student Name: Muhammad Ahsan
Resources currently borrowed: 0
Resources returned to date  : 0
Books reserved          : 0
Pending Fines           : $0
Current Account Balance : $500

========================
    Borrowing History
========================
No borrow history found for this user.

========================
      Reservations
========================
No reservations found for this user.
====================================

  Press Enter to continue...
```

CHAPTER 7: FUTURE EXPANSIONS

1. GUI Interface — Replace the console menu with a graphical interface using Qt for better usability.

2. Database Integration — Replace text files with SQLite for concurrent access and better data reliability.

3. Automatic Fine Reminders — Send due date reminders to students via email or SMS.

4. Search Improvements — Add partial title matching and case-insensitive search.

5. ISBN Lookup API — Auto-fill book details by querying an online ISBN database.

6. Overdue Report — Generate a daily report of all overdue books for the librarian.

7. Multi-copy Tracking — Track individual copies of the same book separately.